Network Security Protocols

IS 2204 - Information Systems Security

Dilanga Harshani

Content adapted from
Prof. Kasun De Zoysa & Mr. Kenneth Thilakarathna

University of Colombo School of Computing

July 10th & 17th, 2026

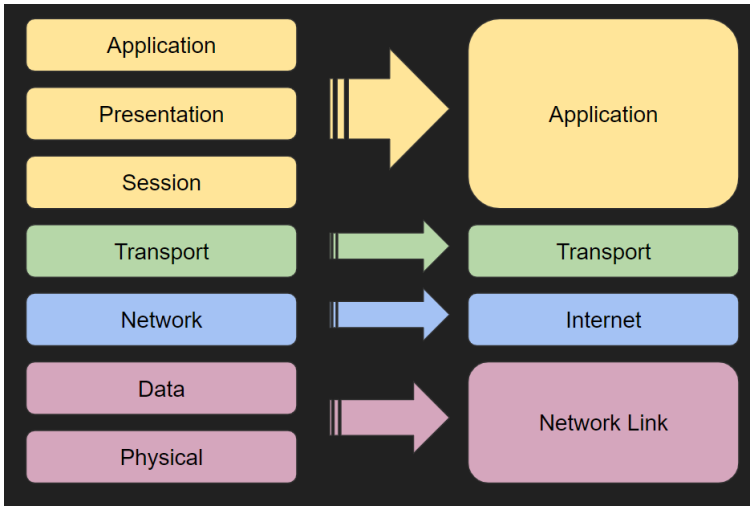
Where You Are

- Introduction to Information Security and CIA Triad
- Symmetric and Asymmetric Cryptography
- Email and Document Security
- Transport Layer Security
- Payment Security and Blockchain
- **Network Security Protocols**

Web Security vs Network Security

What does TLS/HTTPS actually protect?

- A single application (the browser) talking to a server
- What about traffic that is not a browser talking to a website?
- Everything below that routing, name resolution, the raw connection itself?



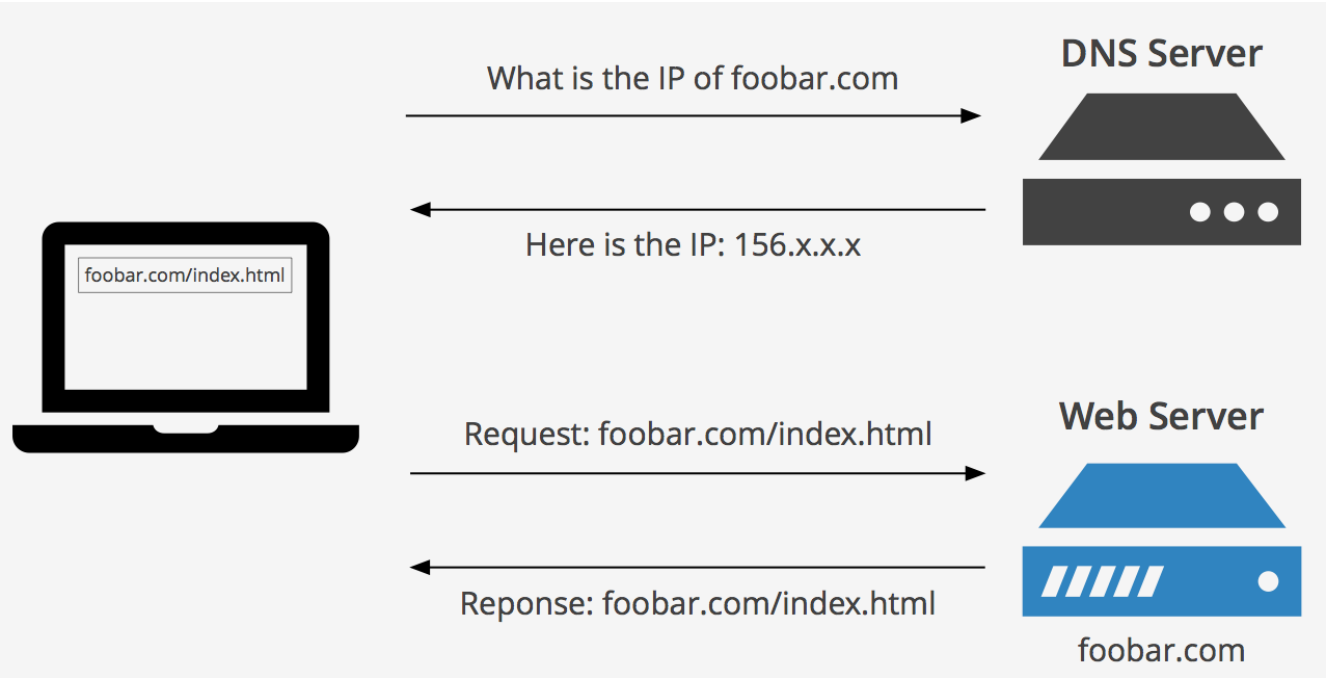
Network Security Roadmap

- **DNS**, a lookup that happens before any app-level security exists
- **SSH**, a protocol that needs its own dedicated secure channel
- **IPSec**, a protection that does not require application awareness

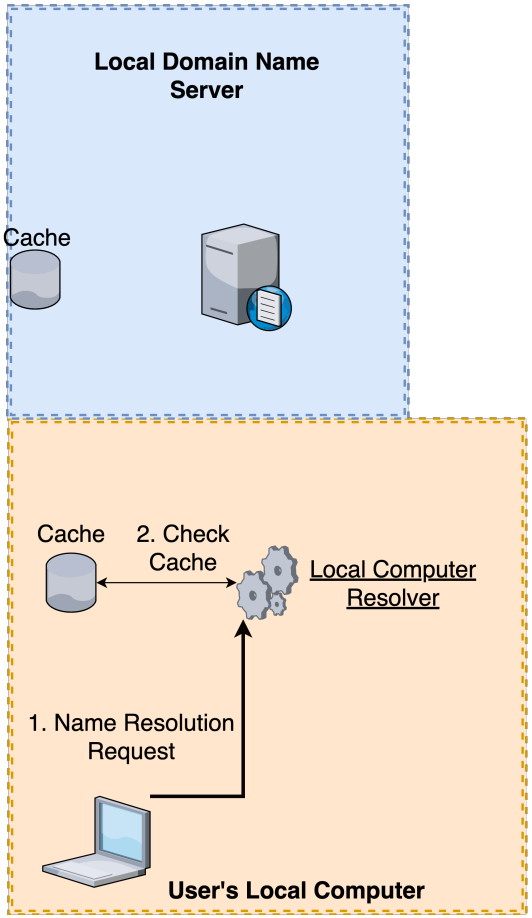
What is DNS?

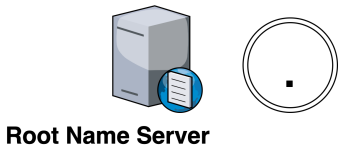
- Humans refer to resources by name, devices in a network require IP addresses
- DNS provides the translation between the two, symbolic hostname into a numeric IP address and vice versa
- DNS stands for Domain Name System, Domain Name Service, Domain Name Server, Domain Name Space

How DNS Works



Domain Name Resolution



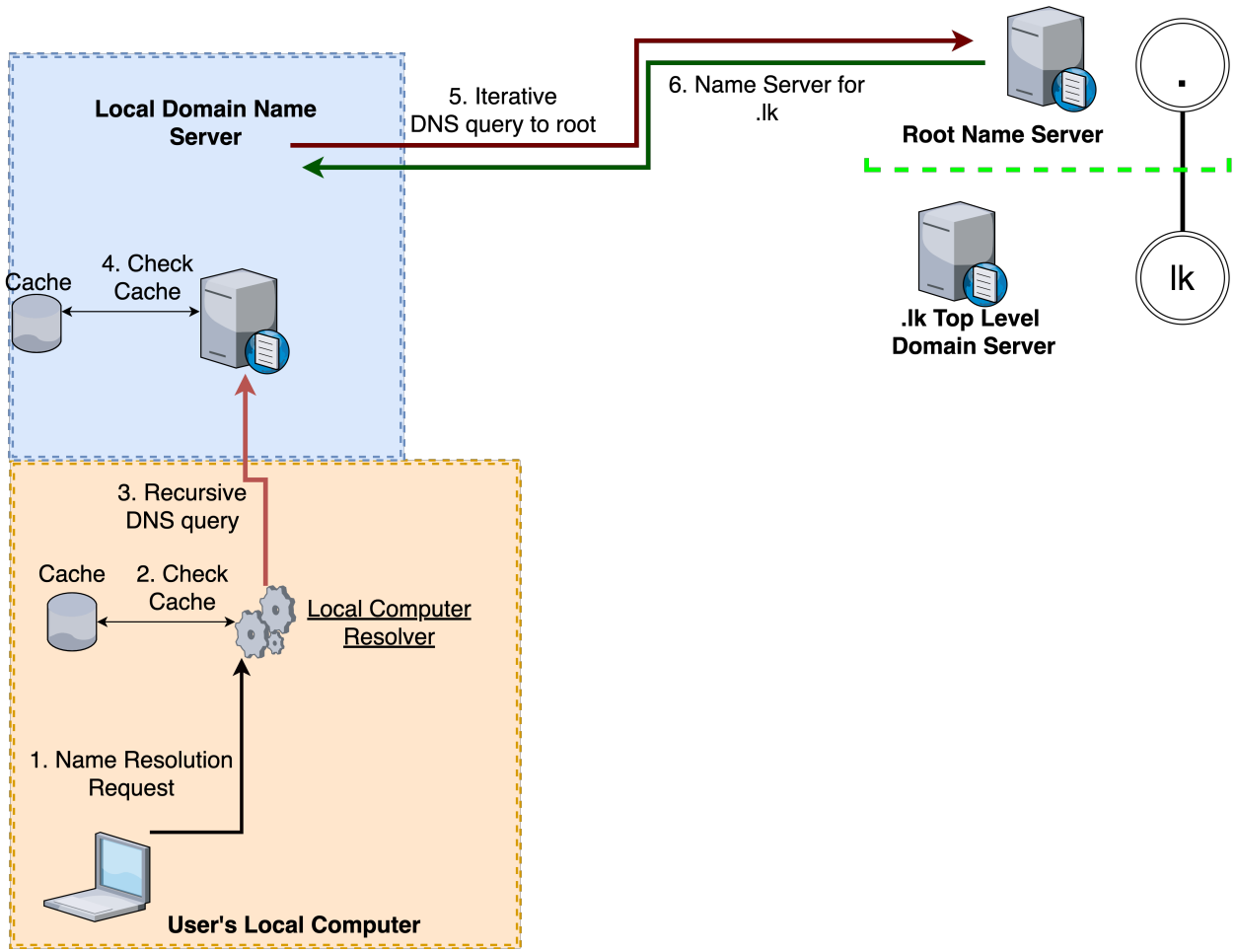


DNS Cache

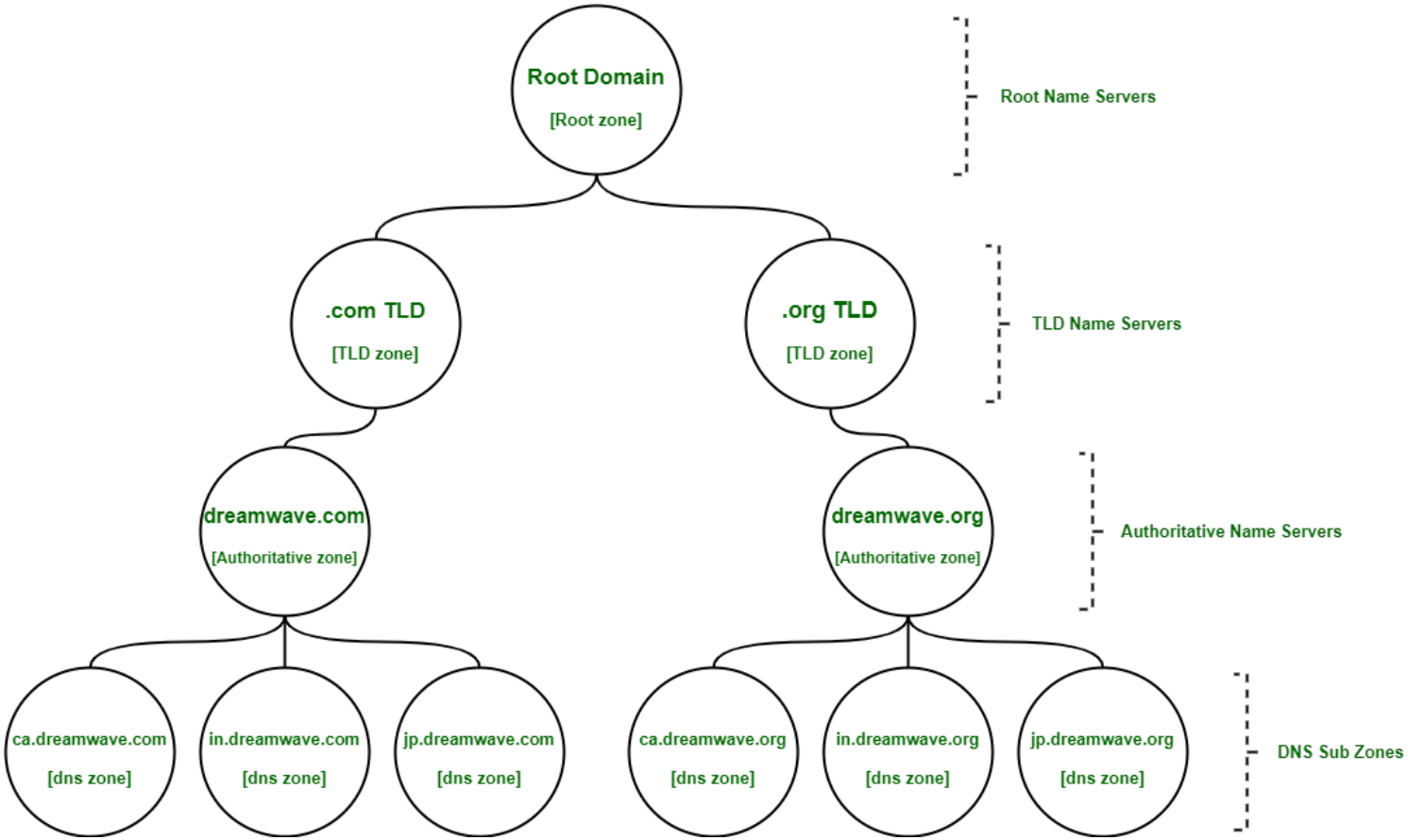
What does DNS Cache do?

- Keep DNS records that are recently accessed
- Temporary and expires after a defined period of time
- Expiration needed to keep the information up to date / synced
- Improve query latency, thus performance

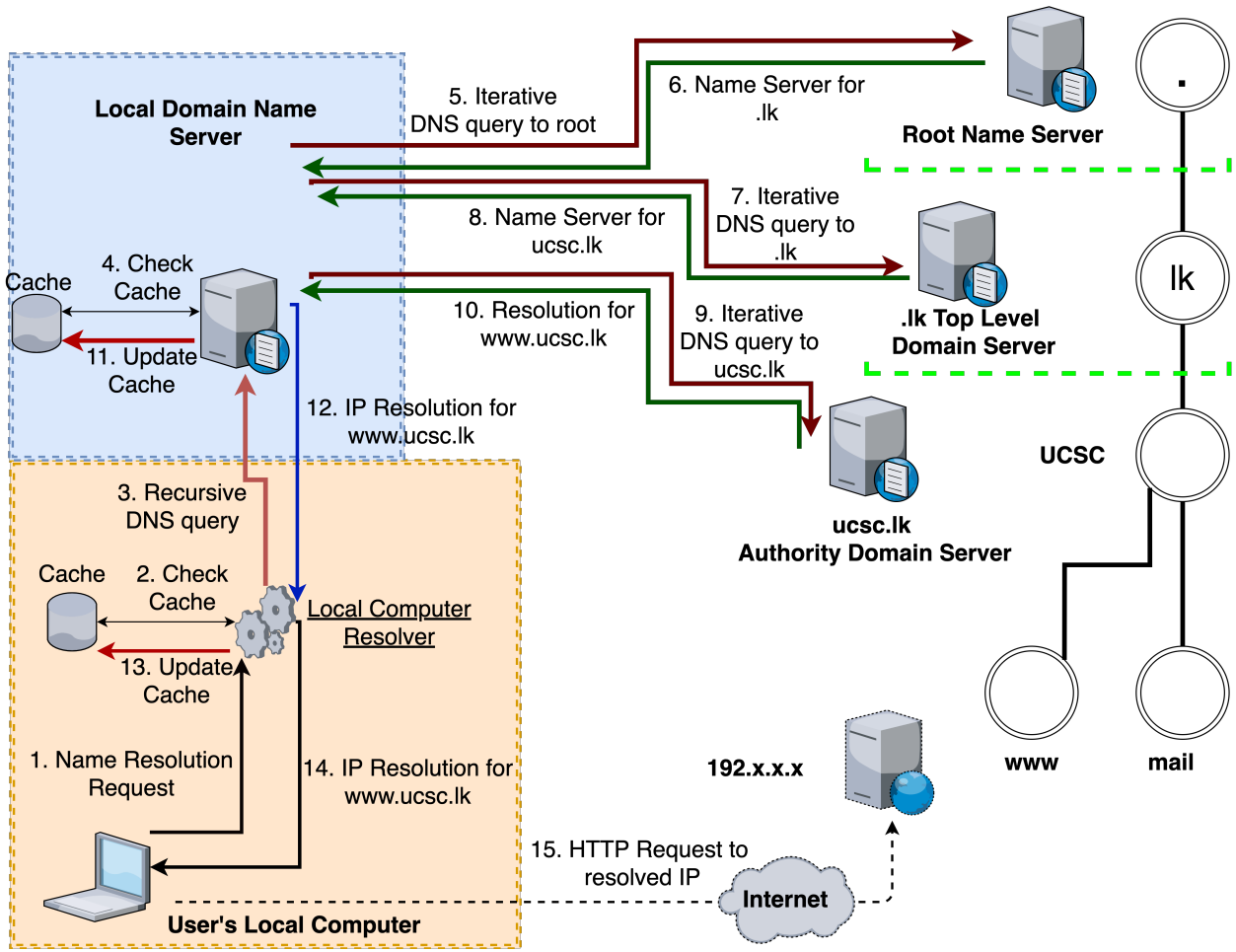
Domain Name Resolution



Domain Name System Zones



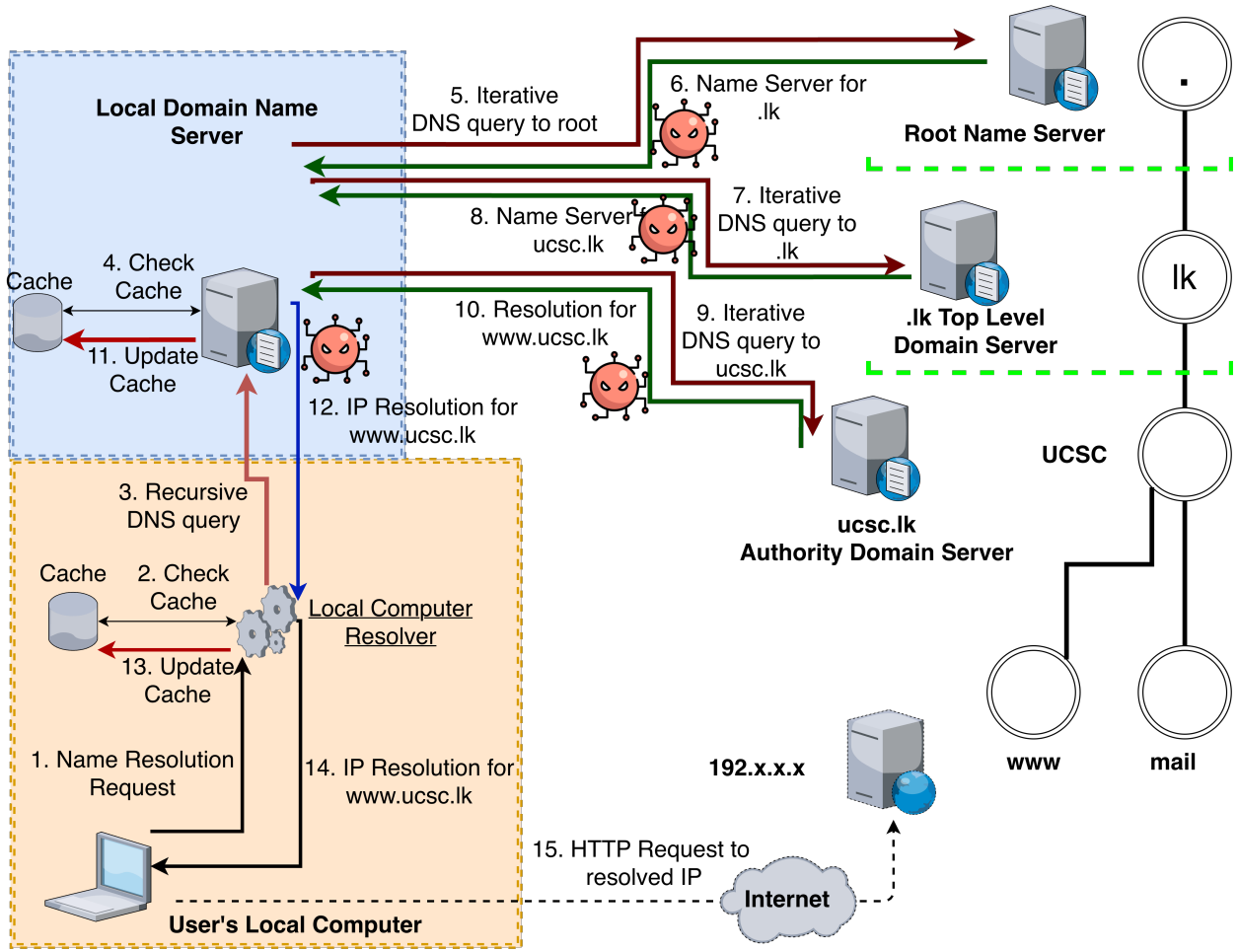
Domain Name Resolution



Where the Vulnerability Lies

- At every stage of this lookup, the response carries no signature
- A forged response which might be faster, guessed, or intercepted will be accepted with the same trust as a genuine one
- No mechanism exists within the protocol to verify the authenticity of a response

Where the Vulnerability Lies



DNSSEC

- A private key signs and the corresponding public key verifies

- Each zone owner has a private (secret) and public key pair
- The zone owner signs its records using its private key
- Any resolver can verify authenticity using the corresponding public key
- If even one bit of data changes, verification fails

Resource Record (RR)

In DNS, information is stored in Resource Records (RRs), which contain various types of data, including

- **A** : Maps a domain name with its corresponding IPv4 address
- **AAAA** : Connects a domain name to an IPv6 address
- **TXT** : Holds text data, often used for verification and policy purposes
- **MX** : Mail exchange record directing email to the correct mail server
- **CNAME** : Provides an alias from one domain name to another

RRSets : An RRset (Resource Record Set) is a group of records of the same type associated with a domain name. For example, all A records for example.com form an RRset.

Three components introduced by DNSSEC

- **Private key**, which is retained securely by the zone owner
- **Public key**, which is published as a `DNSKEY` record
- **Signature**, which is published as an `RRSIG` record

Establishing trust in the Public Key

- Signature verification is only meaningful if the public key itself is authentic
- Manual verification of every zone's key individually does not scale
- A chain of trust is required

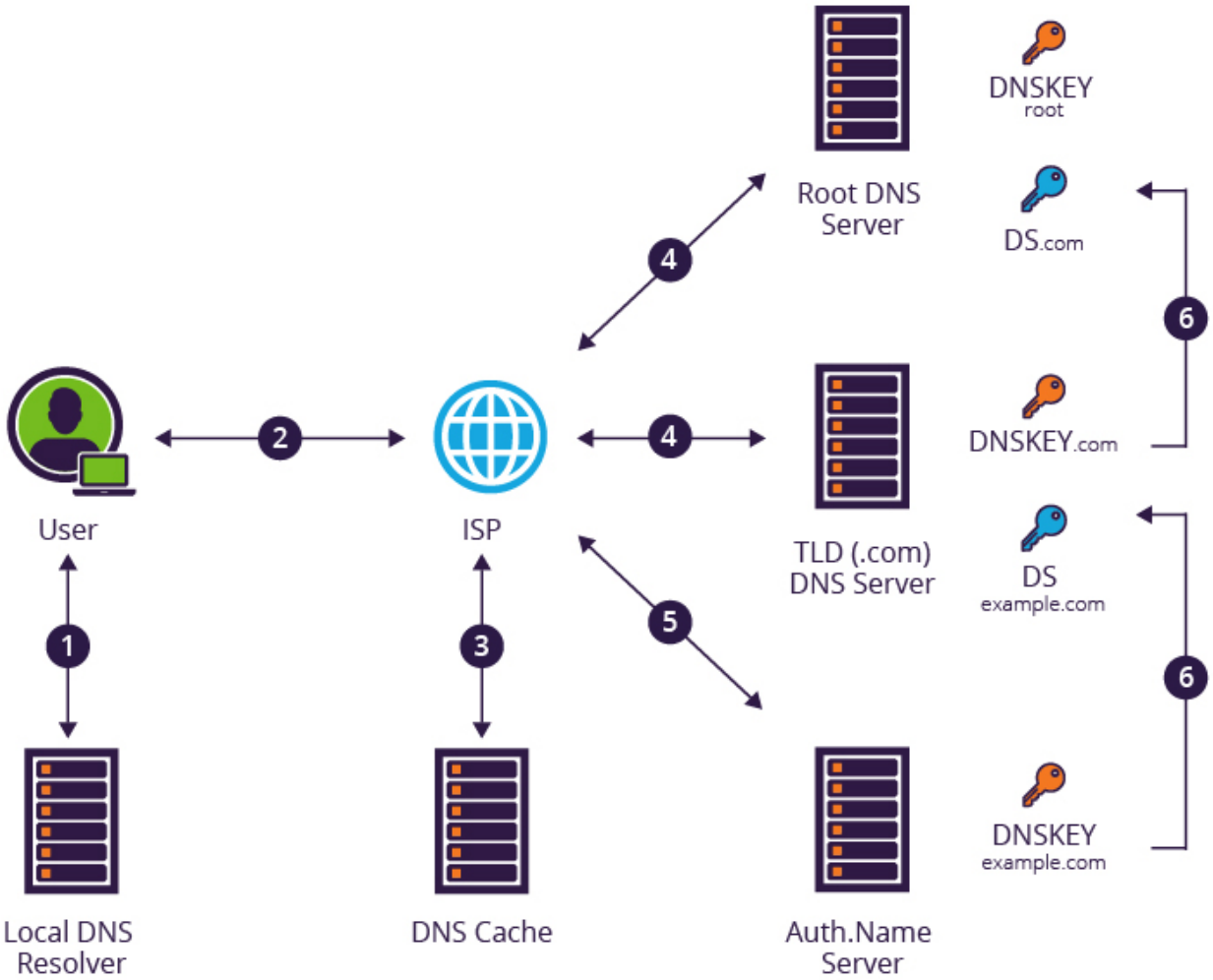
Two Keys, Two Roles

- **ZSK (Zone Signing Key)**
  - Used to sign the routine records and it is rotated frequently
- **KSK (Key Signing Key)**
  - Used to sign the DNSKEY record itself, certifying the ZSK and it is longer-lived

Chain of Trust

- The root key is the single key trusted manually, which is the the trust anchor
- Each parent zone certifies its child zone's key via a **DS record**
- A resolver traverses this chain to verify any individual record
- Break any link, and validation fails (the resolver treats data as untrusted)

Chain of Trust



Secure Shell (SSH)

- Earlier remote login tools (telnet, rlogin, rsh) transmitted passwords in plaintext
- SSH was developed directly in response to a password-sniffing attack (1995)
- Provides a secure channel for remote login and other network services over an untrusted network

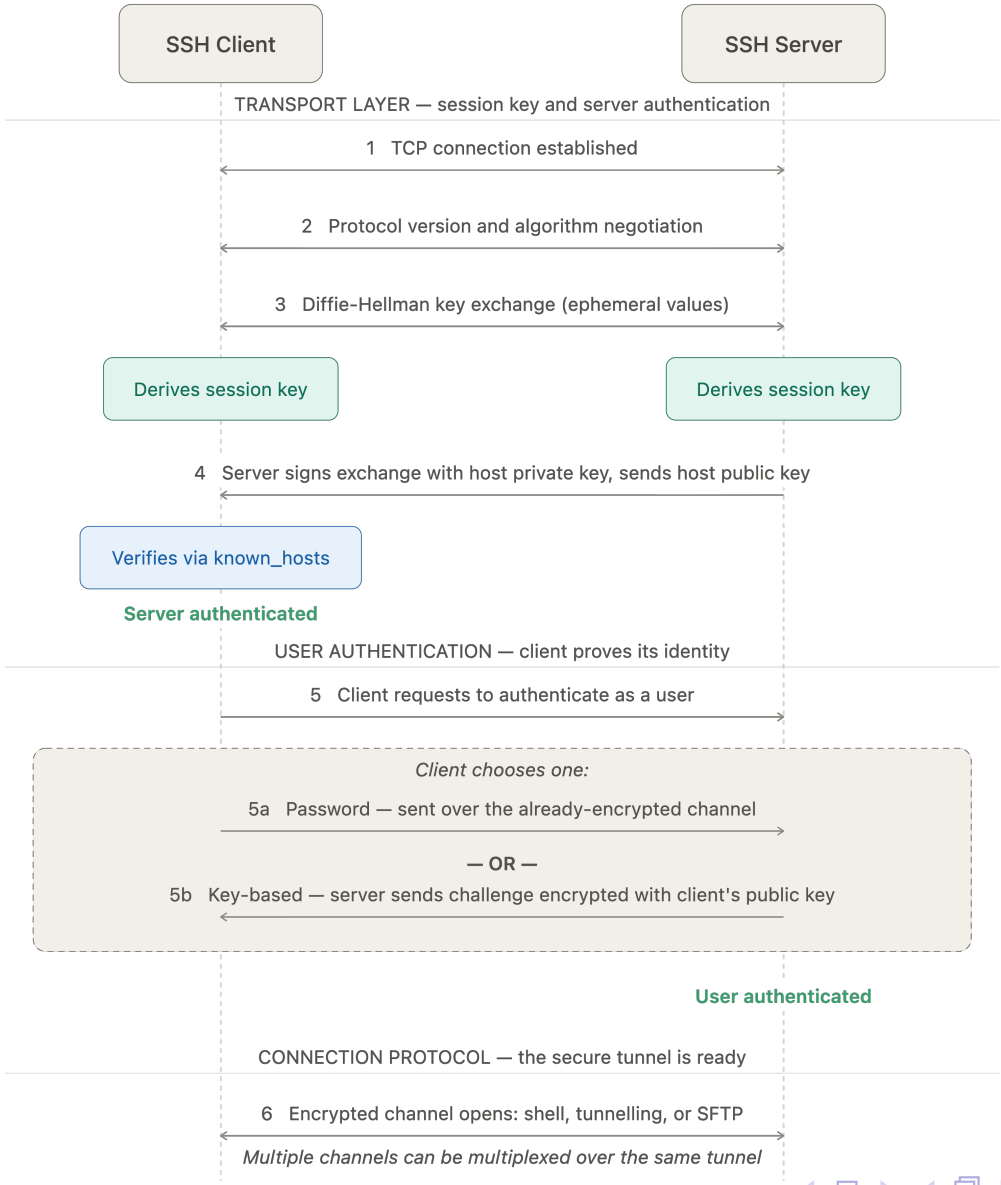
Security Guarantees Provided by SSH

- Confidentiality of the communication channel
- Integrity of the communication channel
- Authentication of the server

SSH Protocol Structure

- **Transport layer** : Server authentication, encryption, integrity
- **User authentication layer** : Verifies the identity of the client
- **Connection protocol**: Multiplexes a single tunnel into multiple logical channels (shell sessions, port forwarding, etc.)

How SSH Works



SSH Connections vs SSH Tunnelling

- **SSH connection (remote login)** : Opens a shell channel, commands run directly on the remote host
- **SSH tunnelling (port forwarding)** : Reuses the same encrypted channel to carry traffic for another application, which gains security it was never designed with
- Both are channel types multiplexed within the same Connection Protocol layer

Secure File Transfer: SCP and SFTP

- Both reuse the same authenticated, encrypted SSH channel, no separate security mechanism required
- **SCP (Secure Copy)** : Simple file copy over SSH, largely superseded due to a limited feature set
- **SFTP (SSH File Transfer Protocol)** : A full file-management protocol (browsing, resuming transfers, permissions), running as an SSH subsystem

Attacks on SSH

- **Man-in-the-middle** : Straightforward if the client has no prior record of the server's public key
- **Denial of service** : Overwhelming the SSH service to prevent legitimate access
- **Covert channels** : An encrypted SSH tunnel can smuggle unauthorized traffic past network monitoring, since the traffic inside is indistinguishable from legitimate SSH use

Reducing Exposure to Automated Attacks

- SSH listens on port 22 by default and this is continuously scanned by automated bots probing for open servers
- Bots attempt credential stuffing and brute-force login against any server found on port 22
- Moving SSH to a non-standard port does not fix the underlying vulnerability, but sharply reduces automated scan traffic and log noise

Password vs Key-Based Authentication

- **Password authentication** : Vulnerable to brute-force and credential-stuffing attacks; strength limited by user-chosen password quality
- **Key-based authentication** : Client proves possession of a private key; the corresponding public key is listed in the server's `authorized_keys`
- Private keys are effectively resistant to brute-force, and can additionally be protected with a passphrase

A Practical Illustration of the Vulnerability

- Prompt: “The authenticity of host ‘X’ can’t be established. Are you sure you want to continue connecting?”
- This asks whether the client already holds the server’s public key
- Accepting without verification permits a man-in-the-middle attacker to intercept the connection

SSH Configuration Files

- `/etc/ssh/` **(server)** : Server configuration and the server's own host keys
- `~/.ssh/id_*` **(client)** : The client's own identity keypair
- `~/.ssh/authorized_keys` **(server)** : Client public keys permitted to authenticate as this user
- `~/.ssh/known_hosts` **(client)** : Fingerprints of servers previously verified by this client

known_hosts and Fingerprint Verification

- No entry for this server yet → First-use prompt
- Entry exists and matches → Connection proceeds silently
- Entry exists but does **not** match → Connection refused by default

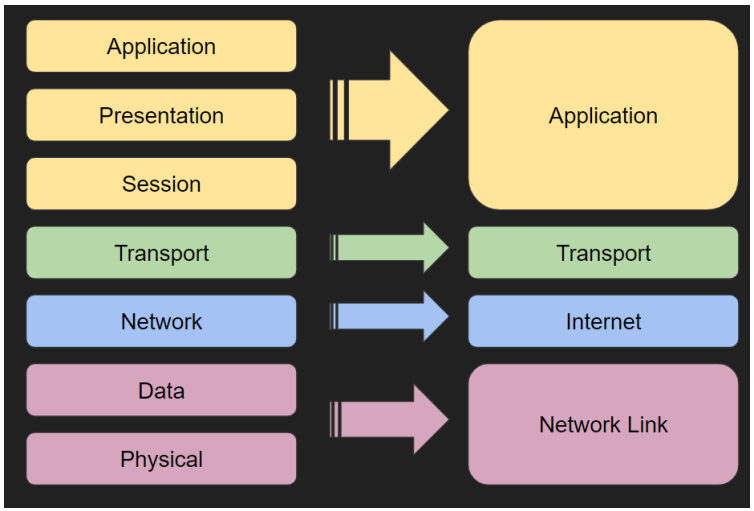
A mismatch indicates either a legitimate host key change (e.g. server reinstalled) or a man-in-the-middle attacker.

A Different Approach: Protection at the Network Layer

- DNS and SSH each address the security of one specific protocol, for connections to servers with no prior relationship
- IPSec instead secures all traffic below the transport layer, for every application, automatically
- Trade-off: IPSec requires an established relationship between endpoints :it does not replace DNSSEC or SSH for one-off connections to unfamiliar hosts

Where IPSec Operates

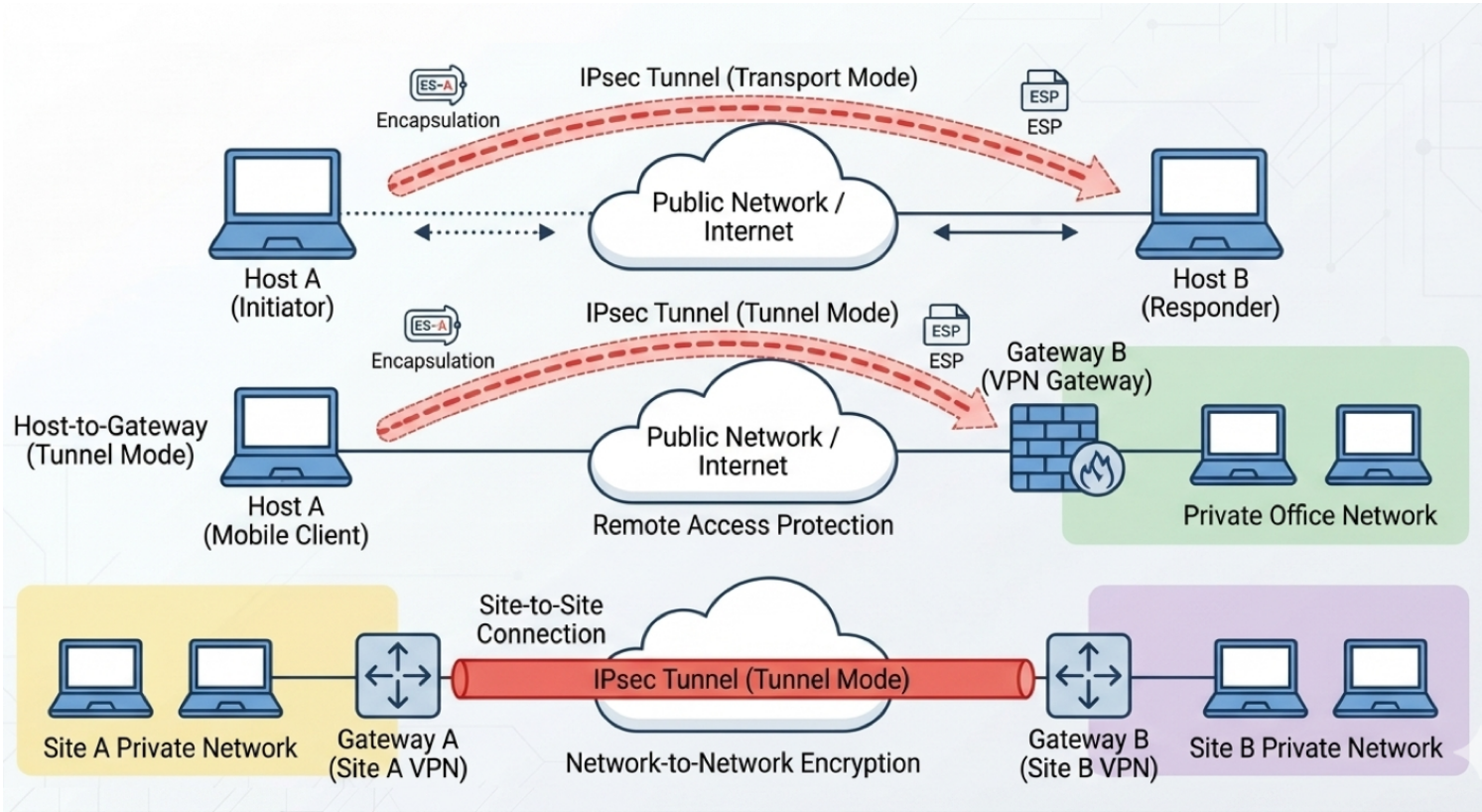
- Operates below TCP/UDP, at the network layer
- Transparent to applications, which require no awareness of its presence
- Protection applies only between endpoints with an established Security Association :not the entire internet



Security Guarantees Provided by IPSec

- Authentication
- Integrity
- Access control
- Confidentiality
- Replay protection (partial)

IPSec Deployments

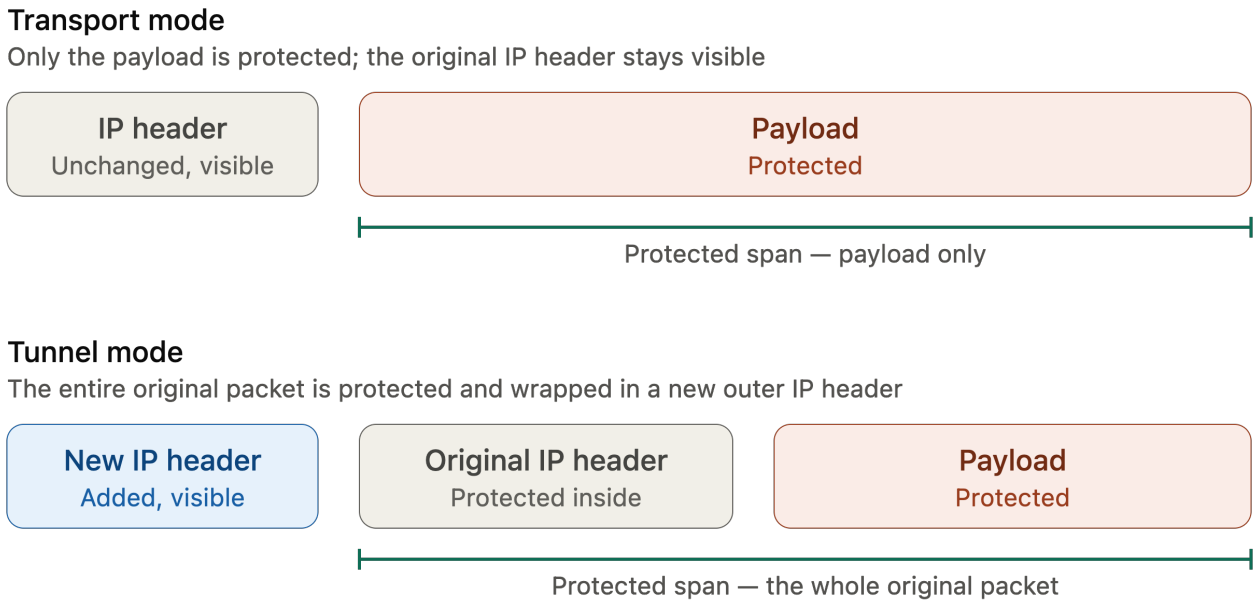


- Host to Host
- Host to Gateway
- Gateway to Gateway (the basis of a Virtual Private Network)

Security Associations

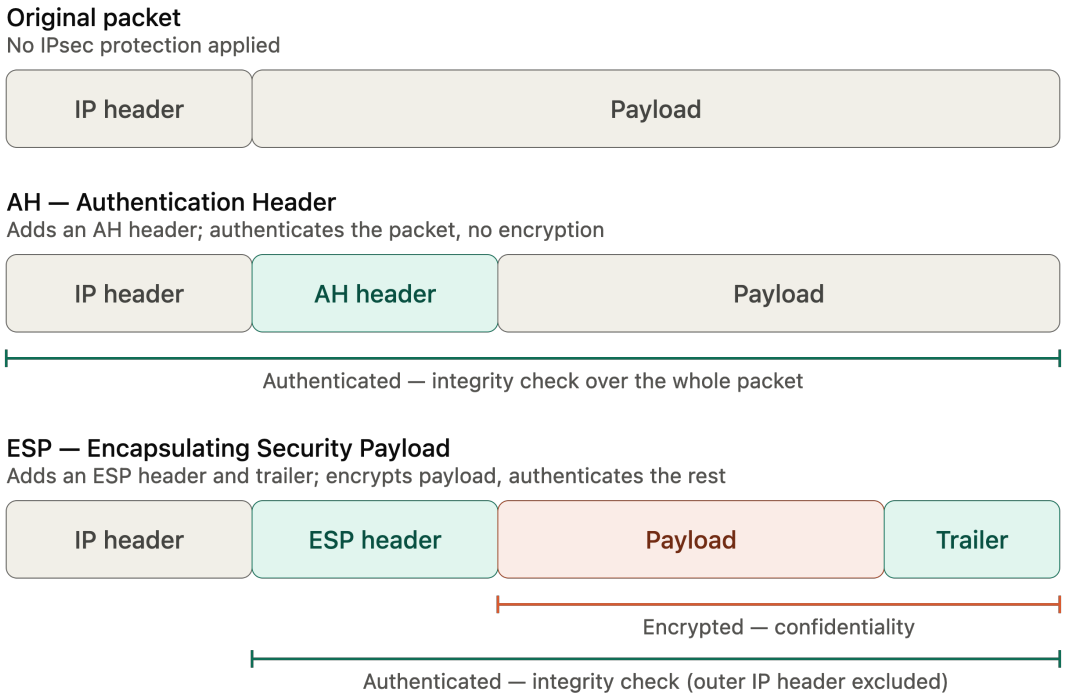
- Prior to establishing protection, both endpoints must agree on *what* to protect and *how*
- This agreement constitutes a **Security Association (SA)**
- An SA is a one-way relationship : bidirectional communication requires two SAs

Modes of Operation : Transport vs Tunnel



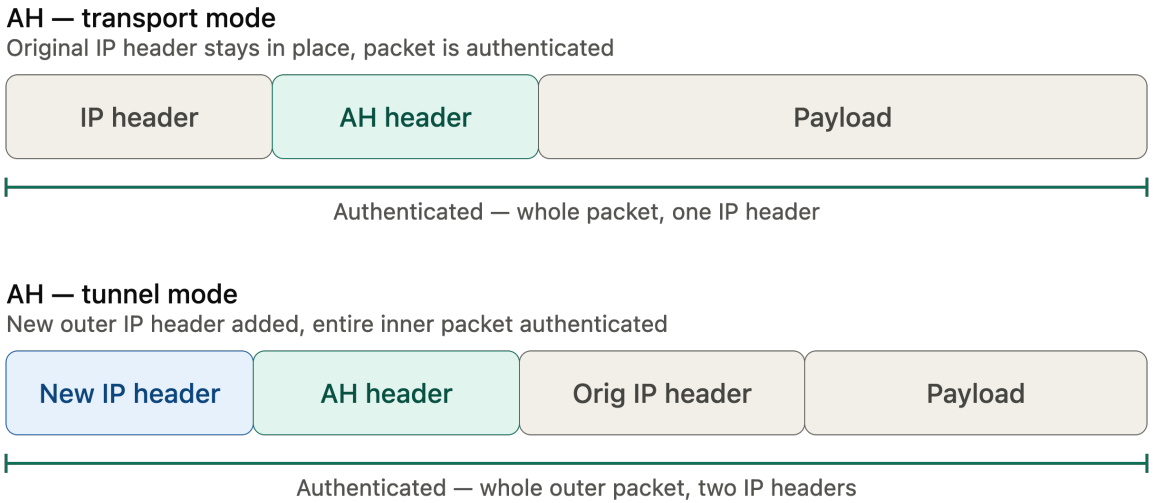
- **Transport mode** : Protects the payload, retains the original IP header; more efficient
- **Tunnel mode** : Protects the entire packet, including sender and recipient identity; more computationally expensive

IPSec Protocols : AH vs ESP



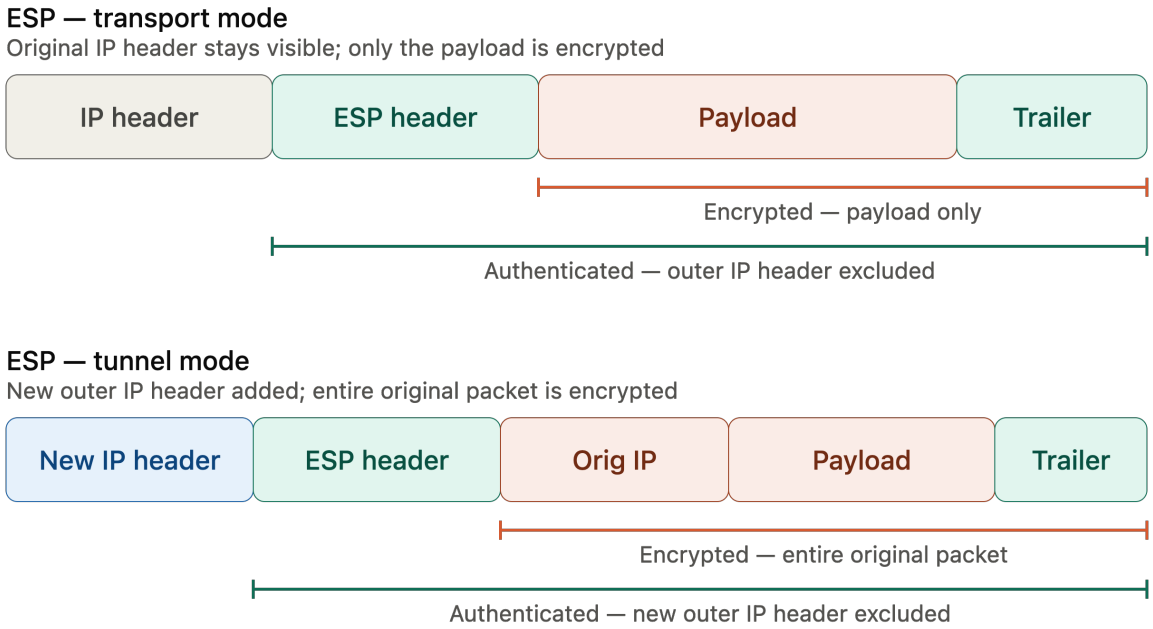
- **AH (Authentication Header)** : Verifies packet authenticity and integrity; does not provide encryption
- **ESP (Encapsulating Security Payload)** : Encrypts the payload and can additionally provide authentication

AH Across Transport & Tunnel Modes



- **AH, transport mode** : Authenticates the payload and selected header fields; no encryption
- **AH, tunnel mode** : Authenticates the entire inner packet and selected outer header fields

ESP Across Transport & Tunnel Modes



- **ESP, transport mode** : Encrypts and authenticates the payload only; outer IP header is not covered
- **ESP, tunnel mode** : Encrypts and authenticates the entire inner packet; new outer header is not covered

Combining AH and ESP

- ESP alone never authenticates the outer IP header, even in tunnel mode
- AH applied on top of ESP extends authentication coverage to the outer header as well
- Provides both full confidentiality (from ESP) and complete tamper-detection (from AH)
- In practice, ESP with authentication enabled is more common; combined use is reserved for higher-assurance deployments

Internet Key Exchange (IKE)

- Manually agreeing on keys and algorithms for every SA does not scale, IKE automates this negotiation between endpoints
- **Phase 1** : Establishes a secure, authenticated channel between the two endpoints
  - The endpoints authenticate each other and agree on a shared secret to protect everything negotiated afterward
- **Phase 2** : Uses that channel to negotiate the security parameters for the actual traffic
  - Agree on the SA(s) that will protect real data :which protocol (AH/ESP), which algorithms, which mode
- The SAs produced by Phase 2 are what AH/ESP actually use once traffic starts flowing

Summary

- **DNS** : Secures a lookup that happens before any application-level security exists
- **SSH** : Builds its own dedicated secure channel for remote login and tunnelling
- **IPSec** : Secures traffic at the network layer, transparently, for every application
- **Transport vs tunnel mode** : Protect the payload only, or the entire packet plus identities
- **AH vs ESP** : Integrity only, or confidentiality with optional integrity
- **IKE** : Automates SA negotiation so AH/ESP have keys and parameters to work with

Any Questions?